CS 4/5789 — Programming Project 3: Balloon Navigation

Alexander Braverman · October 14, 2026

All three methods were implemented in `DP.py`, `VI.py` and `Qlearning.py`; `python test.py` passes every check. All figures below were produced by `python visualize.py --experiment all --save-dir figures` with the default settings (seed 0). Throughout, the start state is $s_{(1,0)}$ and the goal is $s_{(5,5)}$. Two routes to the goal matter for almost every result:

- **Risky route (7 steps):** DOWN, DOWN along the bottom row (two −5 cells), then UP column 5. Undiscounted return −5−5−1−1−1−1+20 = **6**.
- **Safe route (14 steps):** UP column 1, across the top, DOWN column 3, then UP column 5. Thirteen −1 steps then +20: undiscounted return **7**.

Column 0 has no rightward wind anywhere, so a balloon that enters it can never leave; it is a trap worth −1 per step forever.

6.1 Dynamic programming (finite horizon)

H	$V^*_0(s_0)$	Route from start
1	−1	goal unreachable
3	−3	goal unreachable
5	−5	goal unreachable
7	6	risky (exactly fits)
9	6	risky
14	7	safe (first horizon it fits)

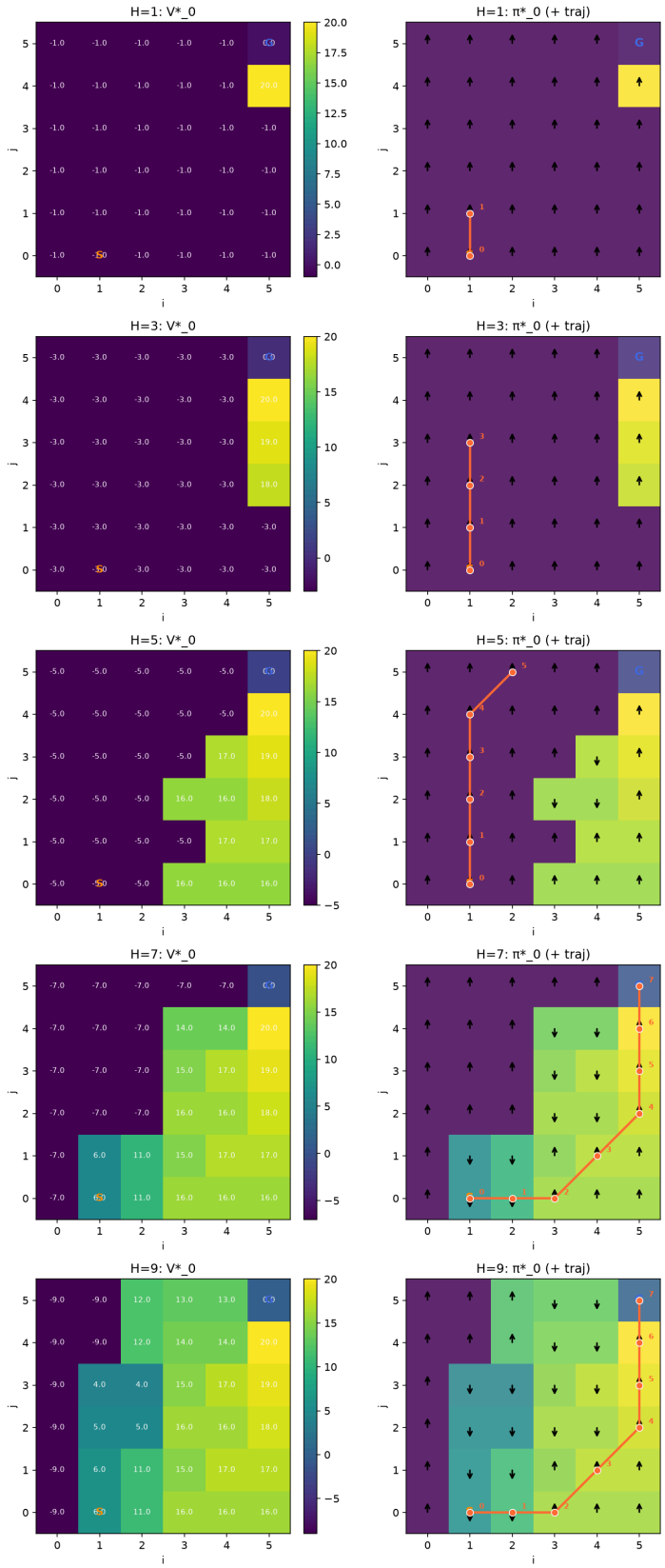


Figure 1. V^*_0 (left) and π^*_0 with the greedy trajectory from the start (right) for $H \in \{1, 3, 5, 7, 9\}$.

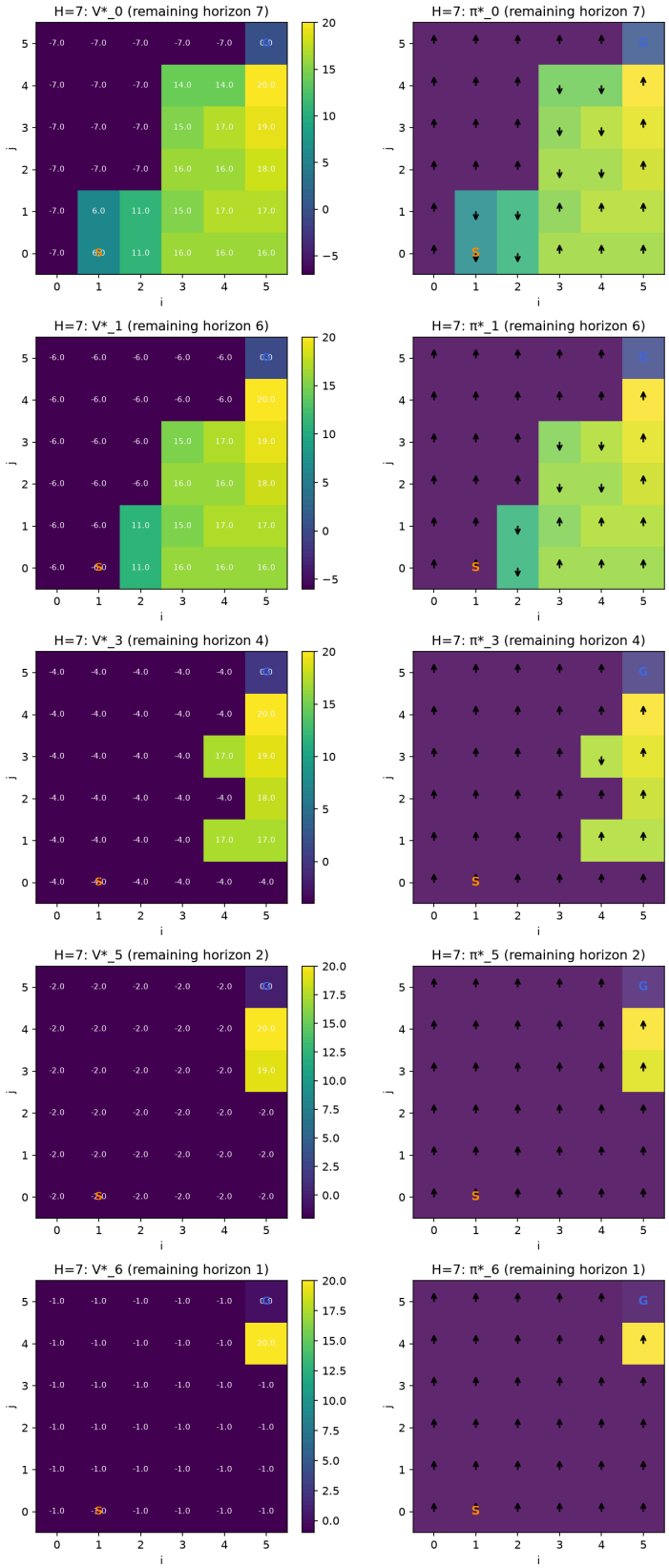


Figure 2. H = 7: time slices (V^*_t, π^*_t) for t = 0, 1, 3, 5, 6, i.e. remaining horizon 7, 6, 4, 2, 1.

6.2 Value iteration (infinite horizon, discounted)

γ	Iterations to tol = 0.01	$V^*(s_0)$	V^*(column 0)	Route from start
0.9	45	−1.66	−9.9	risky
0.99	460	5.30	−99.0	safe
0.999	4607	6.82	−990.1	safe

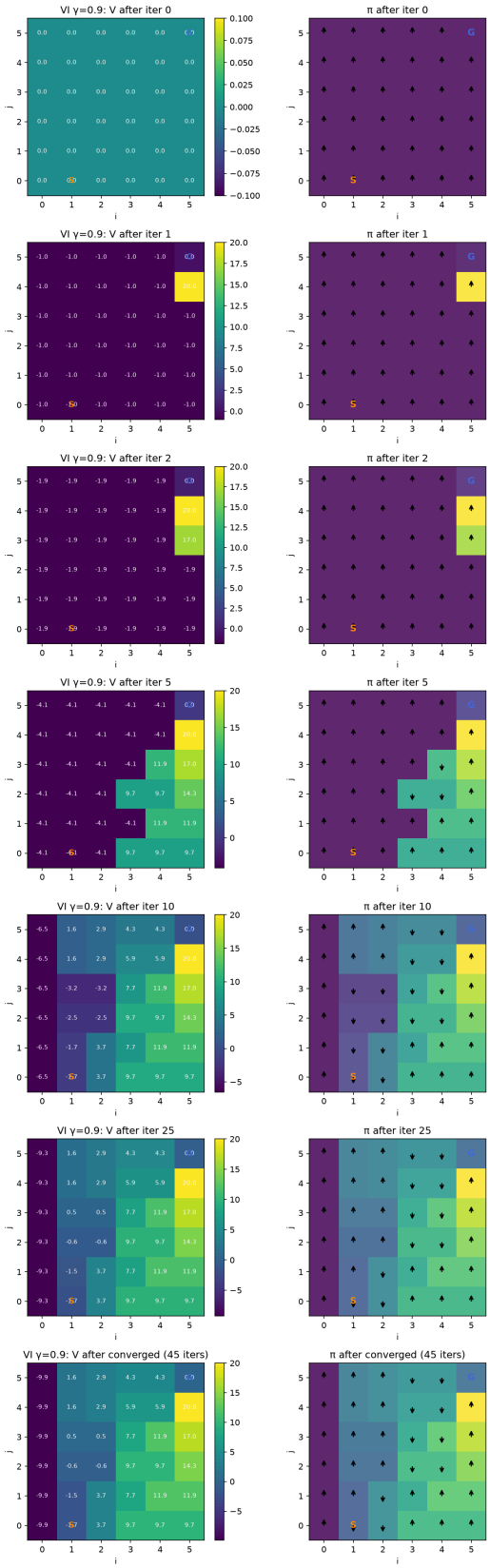


Figure 3. γ = 0.9, starting from Q = 0: V and greedy π after 0, 1, 2, 5, 10, 25 iterations and at convergence (45). Value information spreads outward from the goal one cell-layer per Bellman update.

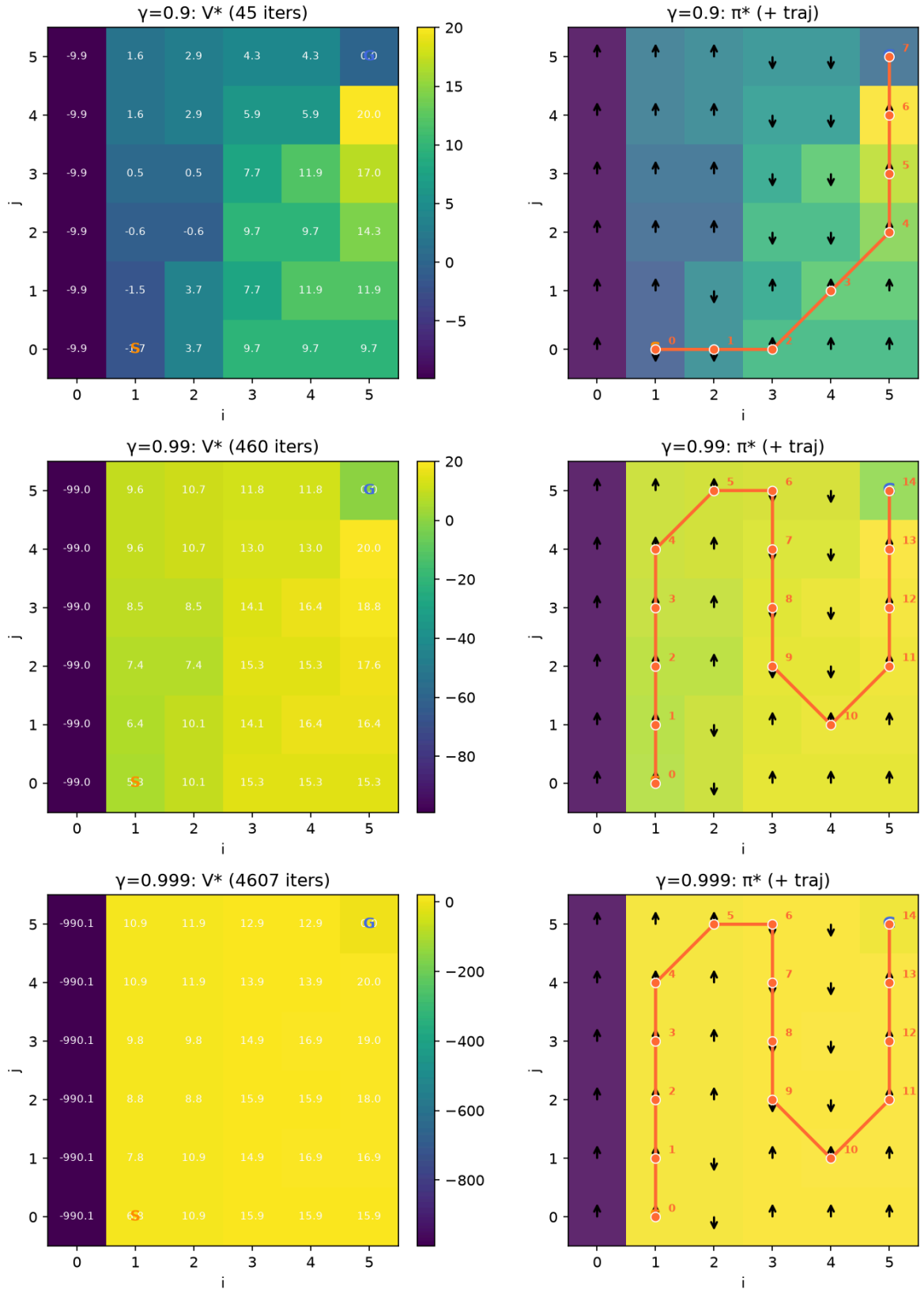


Figure 4. Converged V* and π* (with greedy trajectory) for γ ∈ {0.9, 0.99, 0.999}.

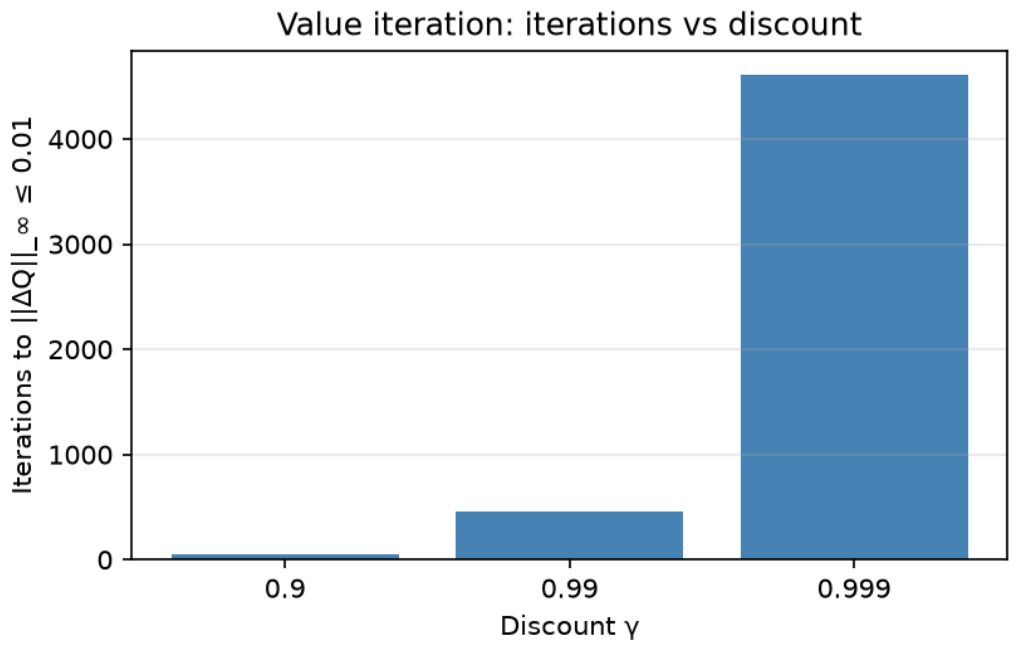


Figure 5. Iterations required to reach the same tolerance versus γ.

6.3 Q-learning

Settings: γ = 0.9 (so Q^*_{VI} is the 45-iteration solution above), 1000 episodes, α = 0.5, Q initialised to 0. Only ε is varied.

ε	final $E_k = \|\|Q_k - Q^*_{VI}\|\|_\infty$ on S_{reach}	final policy agreement A_k	avg return, last 50 episodes	greedy route from start
0	14.30	95%	7.00	safe (not optimal for γ=0.9)
0.1	3.11	100%	4.80	risky (= VI)
0.5	0.00	100%	−4.82	risky (= VI)

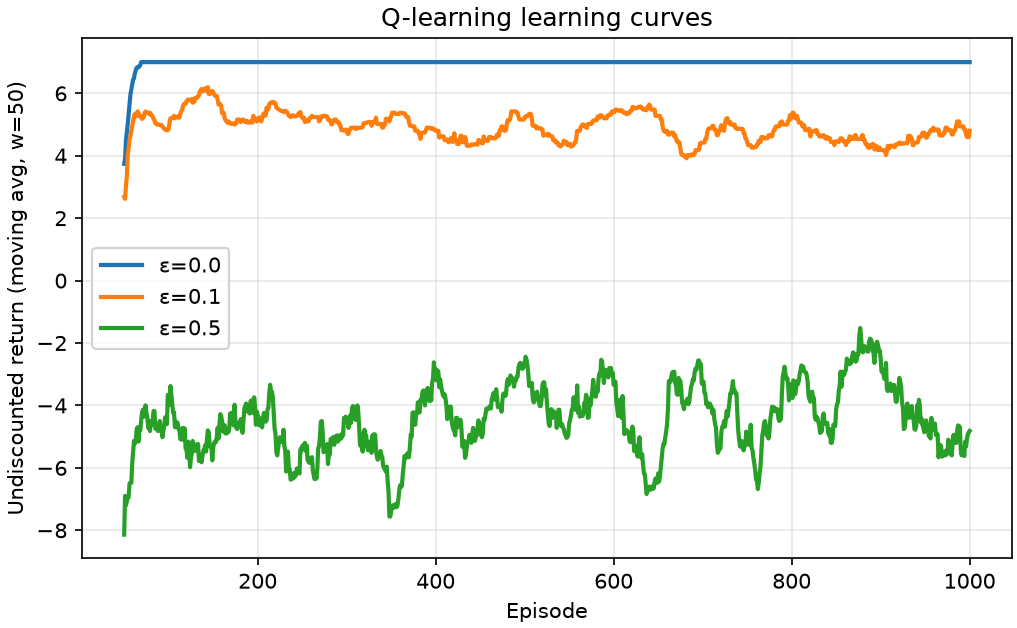


Figure 6. Undiscounted return per episode (moving average, window 50) during training.

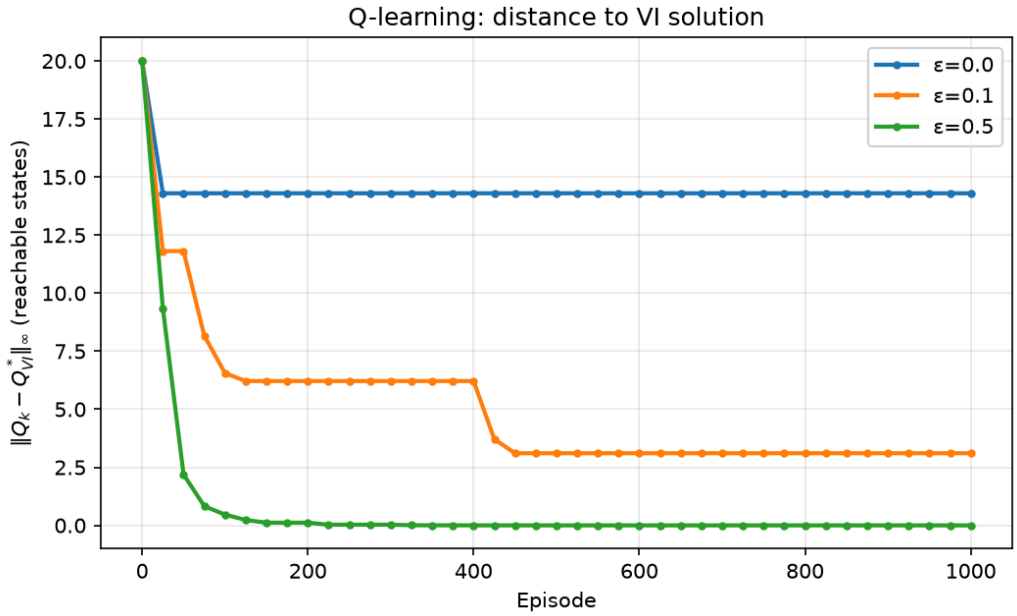


Figure 7. E_k, the max-norm distance between the learned Q and Q^*_{VI} over the reachable states.

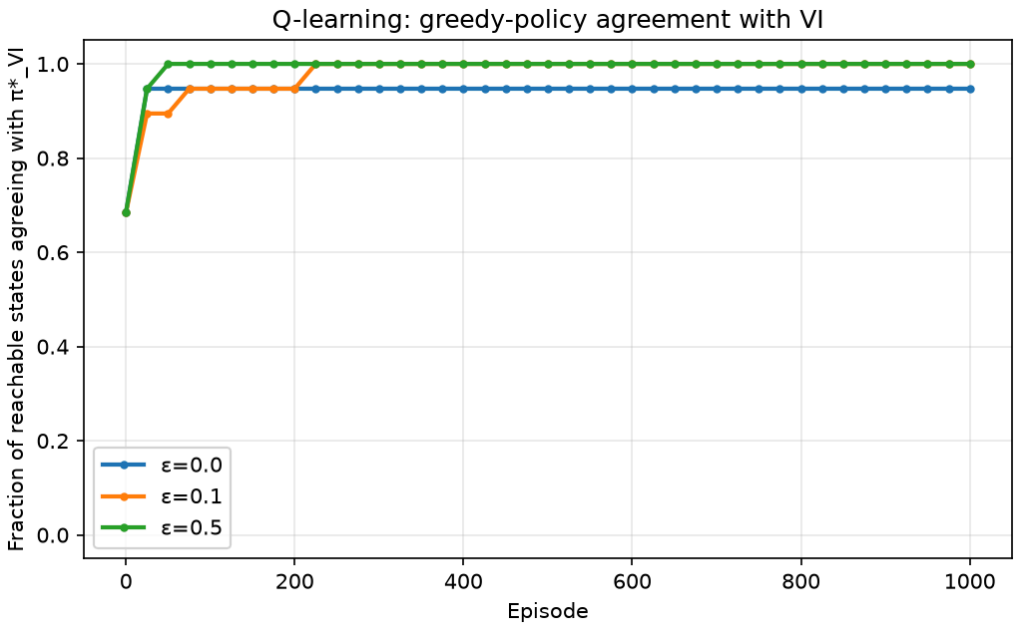


Figure 8. A_k, the fraction of reachable non-goal states whose greedy action matches π^*_{VI}.

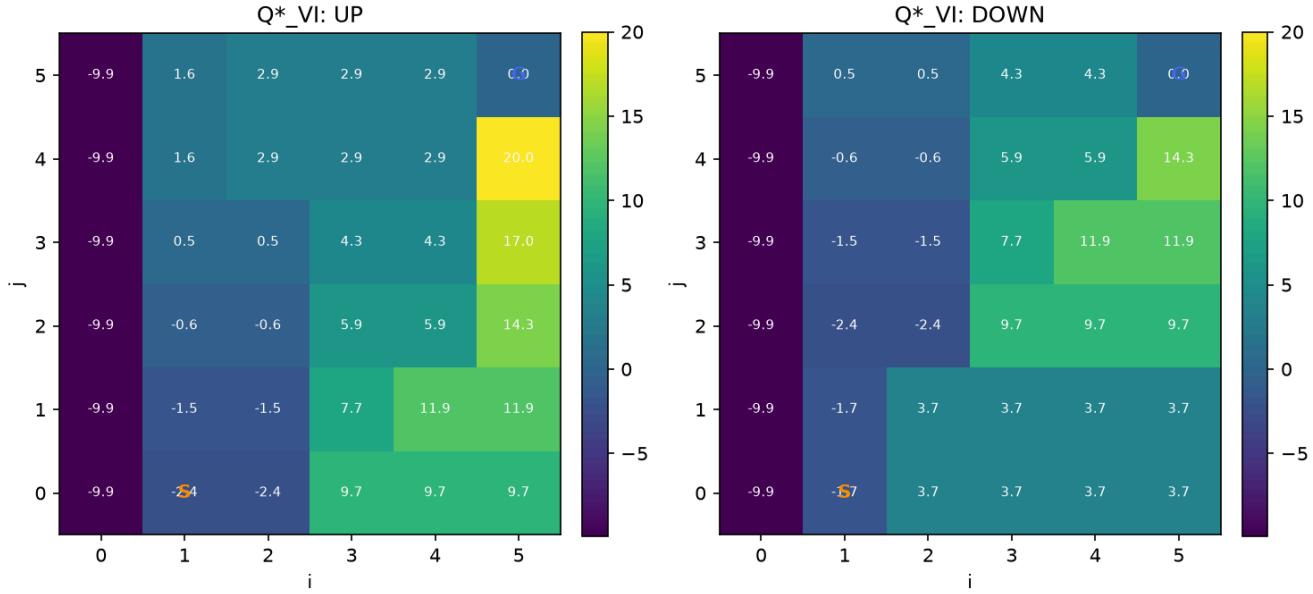


Figure 9. Q^*_{VI} for $\gamma = 0.9$, the reference the learned Q-functions are compared to.

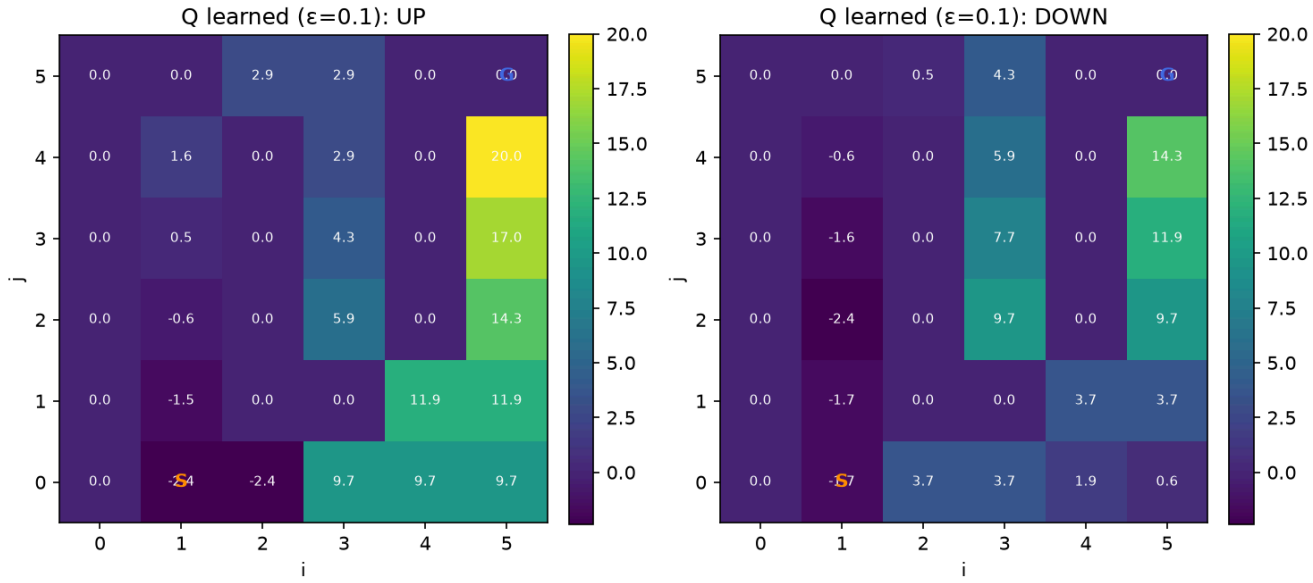


Figure 10. Q learned with ε = 0.1 after 1000 episodes.

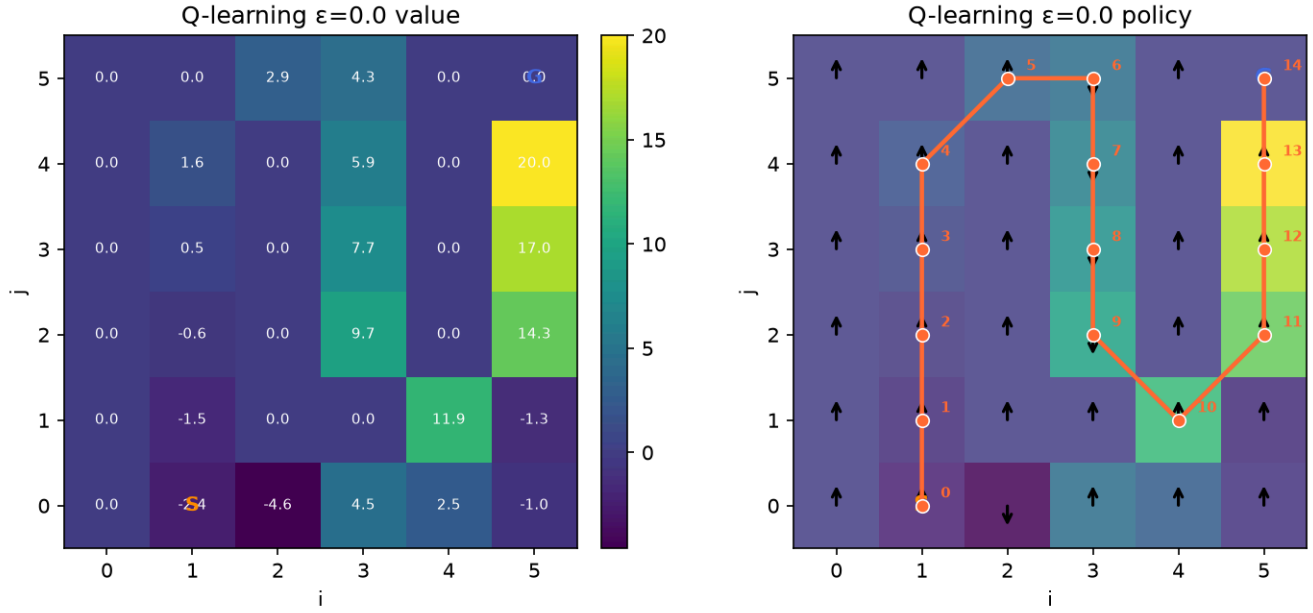


Figure 11. Learned greedy policy, ε = 0.

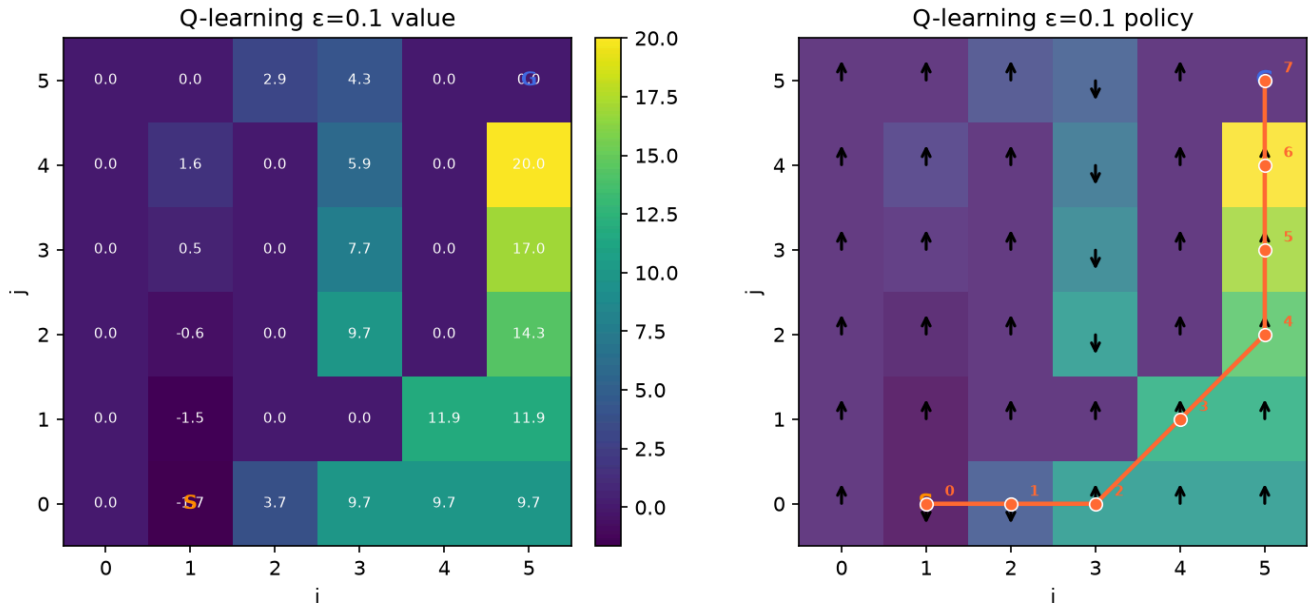


Figure 12. Learned greedy policy, ε = 0.1.

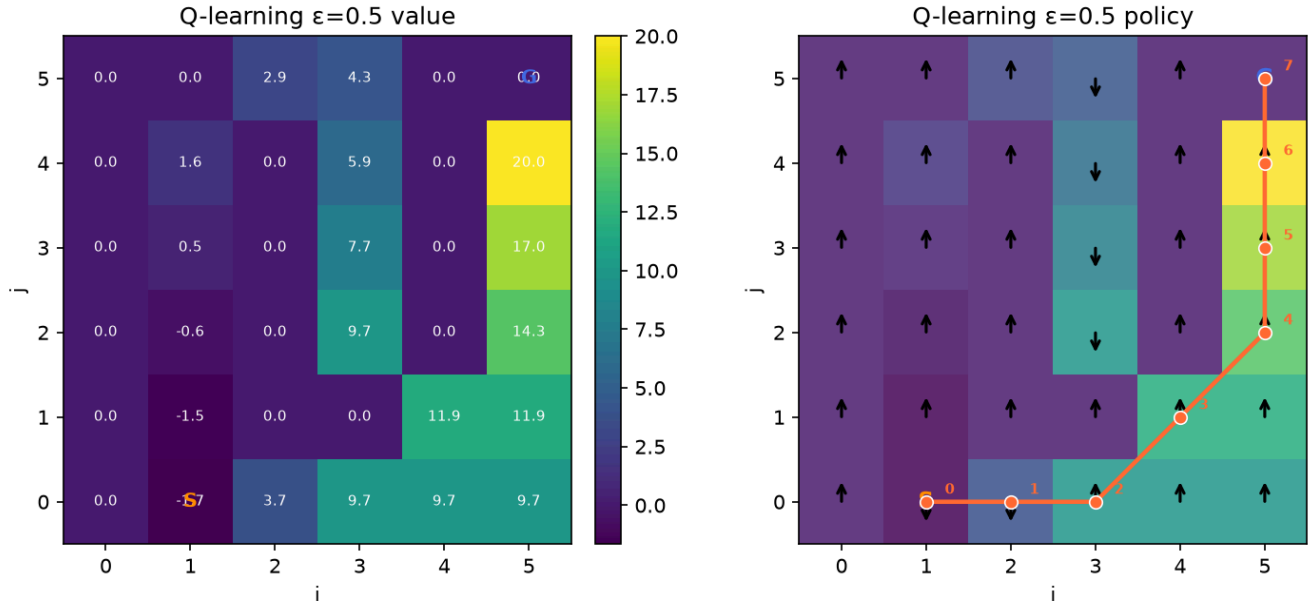


Figure 13. Learned greedy policy, ε = 0.5.

7. Discussion

1. Finite horizon and time dependence.

The shortest path from the start to the goal takes 7 steps, so for $H \in \{1, 3, 5\}$ the goal is unreachable from s_0 and $V^*_0(s_0) = -H$ exactly: the balloon pays −1 per step no matter what, and the only thing the policy can still do is avoid the −5 cells, which is why it points UP away from the bottom row. As H grows, the set of states with positive value spreads outward from the goal by one more "layer" of reachability per unit of horizon (Figure 1), and the value at a state is simply the best total reward collectable in the remaining steps, so V^*_0 is non-decreasing in H. At $H = 7$ the risky route fits exactly and $V^*_0(s_0)$ jumps from −5 to 6; it stays at 6 for $H = 9$ because the better safe route needs 14 steps, and only at $H = 14$ does the policy switch to it ($V^*_0(s_0) = 7$). The time slices in Figure 2 show the same state getting different actions as time runs out: at $t = 0$ the start state goes DOWN to begin the risky route, but at $t = 1$ (only 6 steps left) the goal can no longer be reached from the start, so $\pi^*_1(s_0)$ flips to UP to avoid the penalty cells. When the remaining horizon is too short, the balloon stops trying to reach the goal and just minimises the cost of the remaining steps.

2. Discounting and value propagation.

γ sets how far ahead the agent effectively looks. The trapped column 0 makes this concrete: its value is $-1/(1-\gamma) = -10, -100, -1000$ for the three discounts. It also changes the optimal policy. The safe route's +20 arrives after 13 steps, so it is worth $20\gamma^{13}$ at the start: about 5.1 for $\gamma = 0.9$ but 17.6 for $\gamma = 0.99$. With $\gamma = 0.9$ that is not enough to beat the shorter risky route, so VI takes the −5 cells ($V^*(s_0) = -1.66$), whereas for $\gamma \geq 0.99$ the long safe route wins (5.30 and 6.82, approaching the undiscounted 7). The number of iterations grows roughly like $1/(1-\gamma)$: 45, 460, 4607 (Figure 5). This is because the Bellman optimality operator is a γ-contraction, so the error shrinks by a factor γ per sweep and reaching a fixed tolerance takes $k \approx \log(\text{tol}/\text{err}_0)/\log(\gamma) \approx \text{const}/(1-\gamma)$ updates. Figure 3 shows the propagation mechanism: starting from Q = 0, each update pushes information about the goal one transition further outward, so after k updates only states within k steps of the goal have "seen" it.

3. Dynamic programming versus value iteration.

For a long enough horizon the two agree: the DP policy π^*_0 for $H = 14$ is identical to the VI policy for $\gamma = 0.999$ (both take the safe route; $V^*_0(s_0) = 7$ versus $V^* = 6.82$), and both share the same structure everywhere: climb column 5, descend columns 3 and 4 to catch the rightward wind at row 1, and treat column 0 as a dead end. They need not be identical, for two reasons. First, the objectives differ: DP maximises an undiscounted sum over exactly H steps, while VI maximises a discounted infinite sum. The horizon is a hard cutoff (a route either fits or it does not, which is why $H = 9$ still picks the risky route) while discounting is a soft, geometric penalty on delay; as $\gamma \to 1$ and $H \to \infty$ these converge, but for any finite H and $\gamma < 1$ they can rank routes differently. Second, the DP policy is time-varying, π^*_t

depends on the remaining steps, whereas the VI policy is stationary; DP's π^*_0 happens to match VI only because at t = 0 the remaining horizon is large.

4. Value iteration versus Q-learning.

Value iteration has the model. Its update $Q(s,a) \leftarrow R(s,a) + \gamma \Sigma_{s'} P(s'|s,a) \max_{a'} Q(s',a')$ uses the exact reward and the exact next-state distribution, so in one sweep it can evaluate every (s,a) pair, including states the balloon would never visit, and it needs no interaction at all. Q-learning only has `reset()` and `step()`. It never sees P or R; it sees one sampled transition (s_t, a_t, r_t, s_{t+1}) at a time, and its target $r_t + \gamma \max_{a'} Q(s_{t+1},a')$ is a single-sample estimate of the expectation VI computes exactly (the learning rate α averages these samples over time). Because the sample comes from actually being in s_t and taking a_t, Q-learning can only update the pair it just visited. To learn Q for every state-action pair it therefore has to physically reach every state and try every action there, which is why it needs many episodes and an exploratory behaviour policy, and why the handout evaluates it only on S_{reach}: unvisited rows of the Q-table stay at their initial value (the untouched cells in Figures 11–13 still show the default UP).

5. Exploration in Q-learning.

More exploration makes the learned Q approach Q^*_{VI} faster and more completely, at the cost of worse return while training. With ε = 0.5 every reachable (s,a) pair is tried often, so E_k falls to 0 and A_k reaches 100% within about 75 episodes (Figures 7–8), but half of the actions are random, so the behaviour return hovers around −5 (Figure 6). With ε = 0.1 the greedy policy matches VI after about 220 episodes, but E_k plateaus at 3.1 because pairs off the greedy path are visited rarely and their values are never refined; the return during training (≈4.8) is close to the optimal 6, degraded only by the occasional random step. With ε = 0 the agent is purely greedy. Starting from Q = 0, ties break to UP, so the first trajectory climbs column 1, finds the safe route, and earns 7; from then on every Q-value along that path is positive and every alternative is still 0, so the agent repeats the same path forever. It never tries DOWN at the start, so it never discovers that under γ = 0.9 the risky route is actually better: E_k is stuck at 14.3, A_k at 95% (the start state disagrees), and the learning curve is a flat line at 7. The flat line looks like the best of the three in Figure 6, but that is the undiscounted return of a suboptimal policy. The problem with ε = 0 is exactly this: without exploration Q-learning can lock onto the first policy that works and has no mechanism to correct it, so it is not guaranteed to converge to Q*.